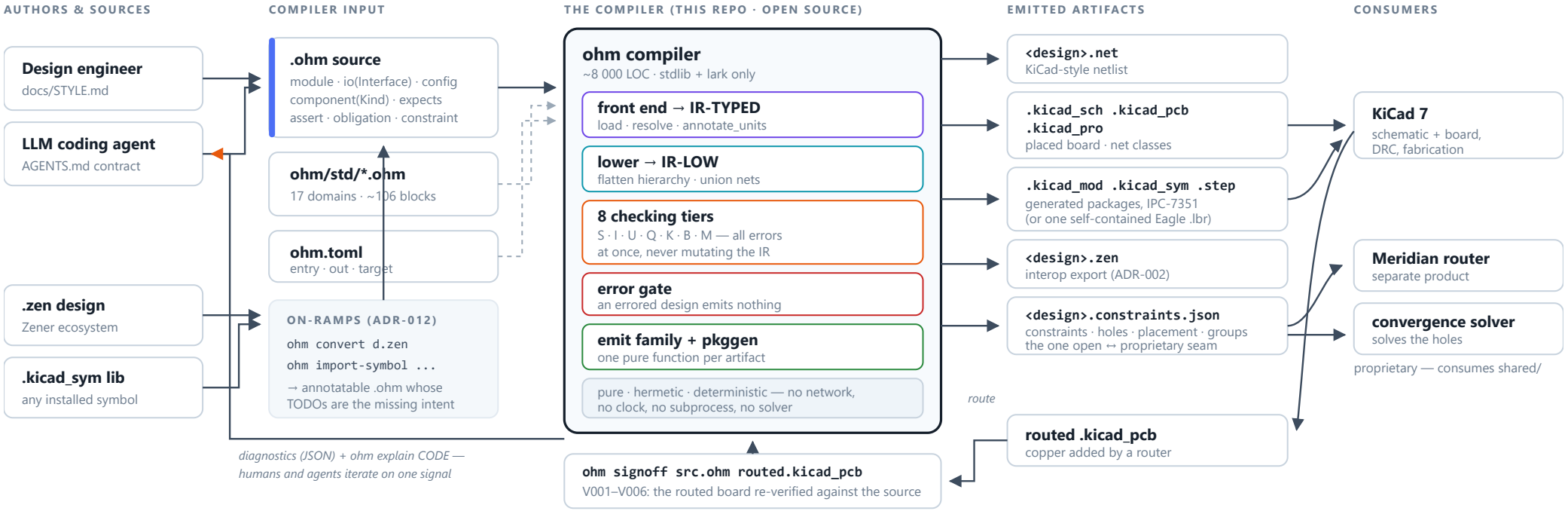
Ohm — an AI-native, physics-aware hardware description language

Ohm is a **compiler**, not a CAD tool. Circuits are written as declared intent — typed interfaces, component `kind` contracts, operating points, bring-up expectations — and a hermetic pass pipeline proves what was declared, deterministically, at compile time. Everything that leaves the compiler (netlist, KiCad 7 project, generated packages, `.zen` interop, constraints JSON) is a pure function of the source: same bytes in, same bytes out, forever. Diagnostics are machine-readable, so humans and LLM agents iterate on the same signal.

SYSTEM CONTEXT — WHO FEEDS OHM, WHAT OHM PRODUCES, WHO CONSUMES IT



CONTENTS

02 Compiler pass pipeline
The driver spine: load, resolve, annotate, lower once, check eight ways, gate, emit.

03 The IR at three levels
AST, IR-HIGH, IR-TYPED, IR-LOW — what each holds and why annotation and flattening are separate.

04 The checking engine
Eight tiers, every diagnostic code, and the escape hatch that keeps waivers auditable.

05 Emit & package generation
One pure emitter per artifact, plus the IPC-7351 footprint and symbol generator.

06 Command surface
Five families of behaviour, project discovery, and the flags that matter.

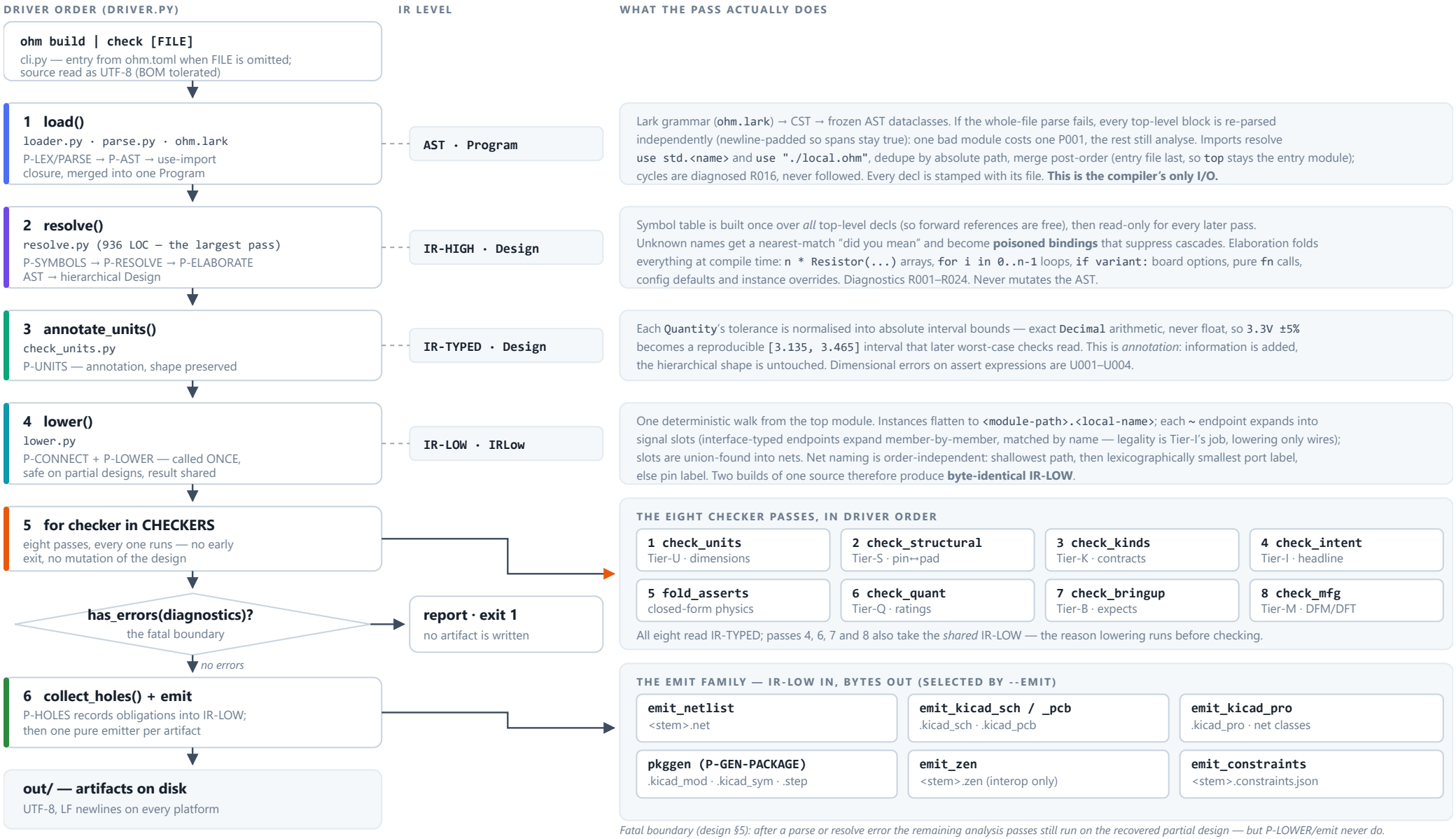
07 Design lifecycle
Scaffold to fabrication, the two feedback edges, and measured results from a real run.

08 Map, invariants, seams
Where each pass lives, what must stay true, and the three interfaces to the outside.

Sources
Read from the repository at v0.1.1: `ohm/`, `shared/`, `docs/blueprint/`, `docs/adr/`, and the v0.1.1 agent run report.

Compiler pass pipeline

One driver (`ohm/driver.py`, 69 lines) orchestrates every pass in a fixed order. Analysis always runs to completion so an agent sees *all* errors at once; lowering happens once and is shared by the checkers and the emitters; nothing is ever written from an errored design.



Fatal boundary (design §5): after a parse or resolve error the remaining analysis passes still run on the recovered partial design — but P-LOWER/emit never do.

The IR at three levels — why lowering exists

A single IR cannot serve parsing, checking and emission well. Ohm keeps three explicit levels (`ohm/ir.py`, all `frozen` dataclasses). IR-HIGH→IR-TYPED is **annotation**: add information, keep the shape. IR-TYPED→IR-LOW is **flattening**: resolve hierarchy, uniquify names, drop module structure. Checkers need hierarchy and types; emitters need flat connectivity — separating them is why both stay short.

resolve · elaborate → *annotate — shape kept* → *lower — flatten*

AST — ast_nodes.py

Syntax, with meaning not yet attached. Produced by the Lark grammar; every node carries a Span and the file it came from, so a diagnostic inside an imported std module points at that file's own line numbers.

`Program(decls)`
UseDecl · InterfaceDecl · ComponentDecl
KindDecl · FnDecl · ModuleDecl
Expr trees kept verbatim for later folding

Recovery: a broken block is skipped with one P001; the surviving decls still resolve and check.

Imports are merged post-order into one Program.

IR-HIGH — hierarchical Design

Mirrors the source: modules contain instances and raw connections. Arrays, loops, `if` variants and `fn` calls are already folded away by elaboration, so the graph is finite, terminating and hermetic.

`Design`
interfaces · components · modules
kinds · fns · top

`Module`
configs · ports · instances · connections
asserts · obligations · constraints

`ComponentDef`
name · package · pins · attrs
padmap · kinds

`Pin`
role · pad · vmax / imax / pmax
voltage / current (operating point)
satisfies · must_connect · tolerant_to

`InterfaceDef · KindDef`
members · requires · parents · required
optional · params · expects

IR-TYPED — the checkers' substrate

The same dataclasses, fully annotated. Nothing is restructured — which is why the checkers can be read as plain traversals, and why a checker that mutated the design would be a bug (asserted).

`Quantity`
value · unit · tol_lo · tol_hi · tol_raw
exact Decimal; tolerance normalised to absolute bounds — the worst-case interval

ADDED AT THIS LEVEL

- a role on every endpoint
- resolved interface member maps
- typed quantities with intervals
- kind closure (declared + ancestors)

READ BY

check_units · check_structural · check_kinds
check_intent · fold_asserts · check_quant
check_bringup · check_mfg · lsp.py

Four of them additionally take the shared IR-LOW, because "is this net driven?" is only answerable on flat connectivity.

IR-LOW — flat, emitter-ready

Hierarchy resolved, names uniquified by a fixed scheme, collections insertion-ordered or sorted by a stable key. This is the object every emitter and signoff consumes — and the determinism guarantee.

`IRLow`
parts · nets · constraints · holes
no modules, no interfaces, no kinds

`PartLow`
ref ("sensor.u1") · designator ("U1")
component · package · attrs

`NetLow · NetNode`
name · nodes · ports · conn_spans
node = (part_ref, designator, pin, pad)
pad falls back to the pin name when unmapped

`ConstraintLow · Hole`
kind · nets/args · params · op · bound_repr
recorded, never solved — they leave via shared/

Net naming is structural and order-independent: shallowest path wins, then the lexicographically smallest port-member label, else the pin label.

SOURCE (EXAMPLES/DIVIDER_LED.OHM, EXCERPT)

```
module DividerLed:
    pwr = io(Power)
    vcc = config(Voltage, default=5.0V)
    r_top = Resistor(value=10kohm +/- 1%)
    c_dec = Capacitor(value=100nF)
    pwr.v ~ r_top.P1
    pwr.v ~ c_dec.P1   pwr.gnd ~ c_dec.P2
    assert divider(vcc, r_top.value, r_bot.value) <= 2.6V
    constraint netclass(pwr.v, "power")
```

AFTER LOWERING — WHAT EVERY EMITTER SEES

`parts` r_top → R1, r_bot → R2, r_led → R3, c_dec → C1
each with its effective package and folded attrs

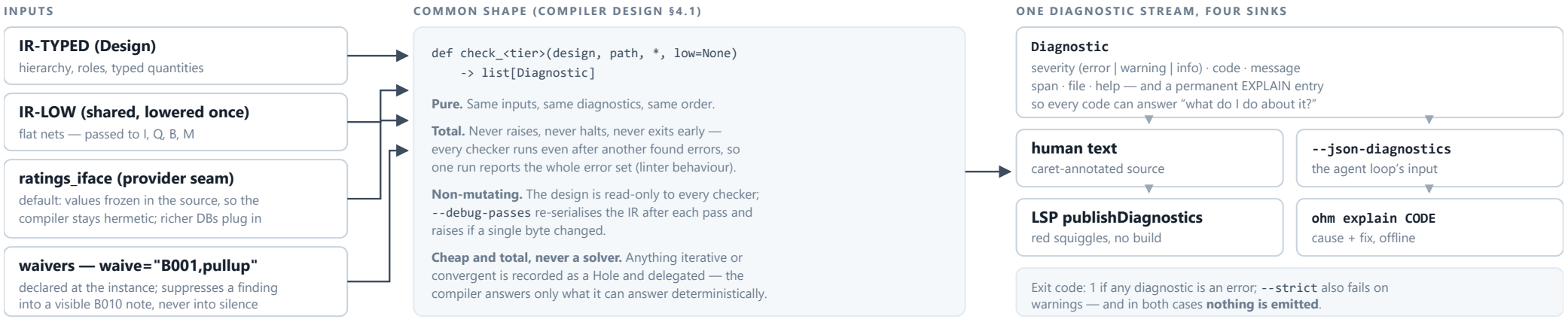
`nets` one net per union of connected slots, named structurally;
`pwr.v` and `pwr.gnd` expand member-by-member into the rail nets

`constraints` netclass(<flat net name>, "power")

`holes` (none here) — `assert` folded at compile time instead: the divider is evaluated over the ±1% intervals and must hold across the whole interval, or it is C001 (certain) / C002 (possible).

The checking engine — eight tiers over declared intent

Generic ERC cannot see protocol-level intent, so the failures that survive DRC and die on the bench (missing pull-ups, floating straps, absent decoupling, unterminated buses, 5 V into a 3.3 V pin) go uncaught. Ohm turns them into compile-time diagnostics. Every check has a declared escape — `waive=` downgrades a finding to an auditable note (B010), never to silence.



TIER	MODULE	WHAT IT PROVES — AND WHAT MAKES IT POSSIBLE	CODES	SEVERITY
P R `parse / resolve`	`parse.py` `resolve.py`	Syntax with per-block recovery; duplicate and unknown names, roles, imports, shadowing, kind and import cycles, array/loop bounds, `fn` purity, foldability of `if` conditions and computed attrs.	**P001 · R001–R024**	error
U `units`	`check_units.py`	Dimensional consistency in exact Decimal: no `3.3V + 10kohm`, no cross-dimension comparison, no incompatible tolerance unit. Tolerances become absolute intervals every later tier reasons over.	**U001–U004**	error
S `structural`	`check_structural.py`	The self-closing pin↔pad loop: no two pins on one pad, no pad outside the generated set, no unmapped pin, every component declares a package. For `generate(...)` packages the pad set is fully known, so the loop closes completely.	**S001–S006**	error
K `kinds`	`check_kinds.py`	Nominal claim, structural check (ADR-008): a part declaring `component X(OpAmp)` must actually satisfy the OpAmp contract over the transitive kind closure — required members bound by role, params present with the right dimension. No declared kind means unasserted, not wrong.	**K001–K005**	error
I `intent` ★	`check_intent.py`	The headline tier. Interface-type mismatches, whole-interface-to-pin connections, multiple drivers on one net, swapped `require`-paired members (SDA↔SCL, D+↔D−), and inputs with no driver — even through a series resistor. Shape rules run per module; driver rules run on the flat nets, where every wiring path lands.	**I001–I003 · I005–I007**	error / warn
C `closed-form`	`closedform.py`	Folds `assert` expressions worst-case over tolerance intervals: an assert passes only if it holds across the whole interval, so certain *and* possible violations both fail the build. Algebraic only — never iterative.	**C001–C004**	error
Q `quantities`	`check_quant.py`	Declared operating points against declared ratings, worst-case: net voltage vs `vmax`, summed draws vs `imax`, $(\Delta V)^2/R$ vs `pmax`, conflicting rails on one net, and a coarse 25 °C + $P \times \theta_{JA}$ thermal bound (ADR-004).	**Q001–Q005**	error
B `bring-up`	`check_bringup.py`	The respin class a correct netlist still contains (ADR-009). Kinds and interfaces declare `expects` predicates — pullup, decoupling, loadcaps, cap, termination, tie — verified over the flat nets: open-drain bus with no pull-up, undefined strap, unpowered `must_connect` pin, over-domain input vs `tolerant_to`, missing crystal load caps, `when endpoint` bus termination, reversed polarised part.	**B001–B009 · B010 (waived)**	error / note
M `manufacturing`	`check_mfg.py`	DFM/DFT lint that opens once MPNs are declared: unorderable BOM lines, one MPN spanning two value/package pairs, valueless passives, rails with no test point or connector, 0402 assembly notes, mixed SMT/THT, missing fiducials, fine-pitch process notes. Advisory by construction — never blocks a build.	**M001–M008**	warn / info
V `signoff`	`signoff.py`	Post-route, against the source rather than the board alone: missing or foreign footprints, pads on the wrong net, incomplete nets, floating copper, point-topology shorts, and `fixed` parts the router moved. Geometric clearance is honestly delegated to `kicad-cli pcb drc` (V005).	**V001–V006**	error / info
Z `interop`	`imp_zen.py`	Structural lints on a flat `.zen` netlist: unparsable Starlark, unknown net vars, out-of-subset constructs, unreadable `pins=`, duplicate names, single-endpoint nets, unconnected parts — plus Z009, the standing notice that a flat netlist carries no intent to check.	**Z001–Z009**	error / warn

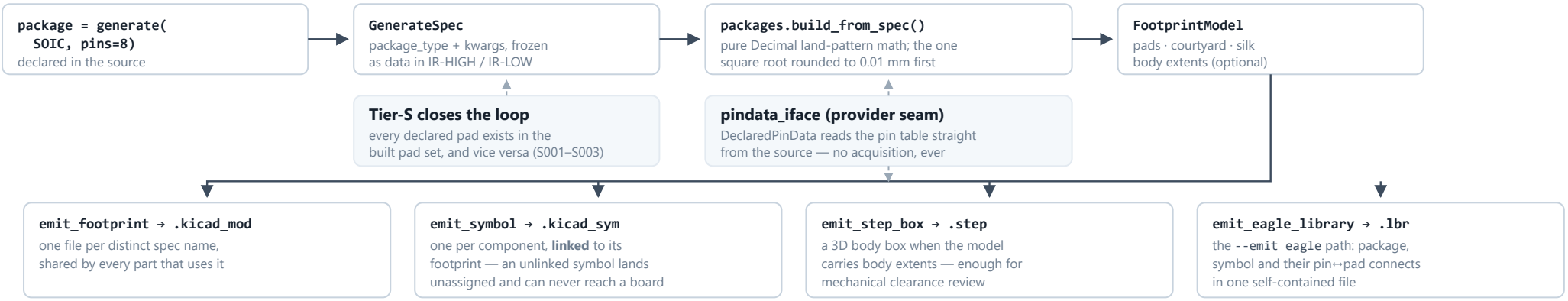
Emit family and package generation

Every emitter is a pure function of IR-LOW — no shared state, no ordering surprises, one artifact each. `--emit` selects a subset; `all` is the default. Nothing here can fail soft: an emitter only ever runs on a design that passed the error gate.

EMITTERS — IR-LOW IN, BYTES OUT



P-GEN-PACKAGE — A PARAMETRIC PACKAGE BECOMES REAL GEOMETRY (ADR-006, IPC-7351)



What an emitter may not do

Read anything but IR-LOW (plus the component table for generated symbols); consult the network, the clock or a subprocess; or depend on another emitter having run. Ordering is fixed upstream in lowering, so an emitter never sorts to "fix" determinism — if it had to, the bug would be in the wrong place.

The stub-footprint decision

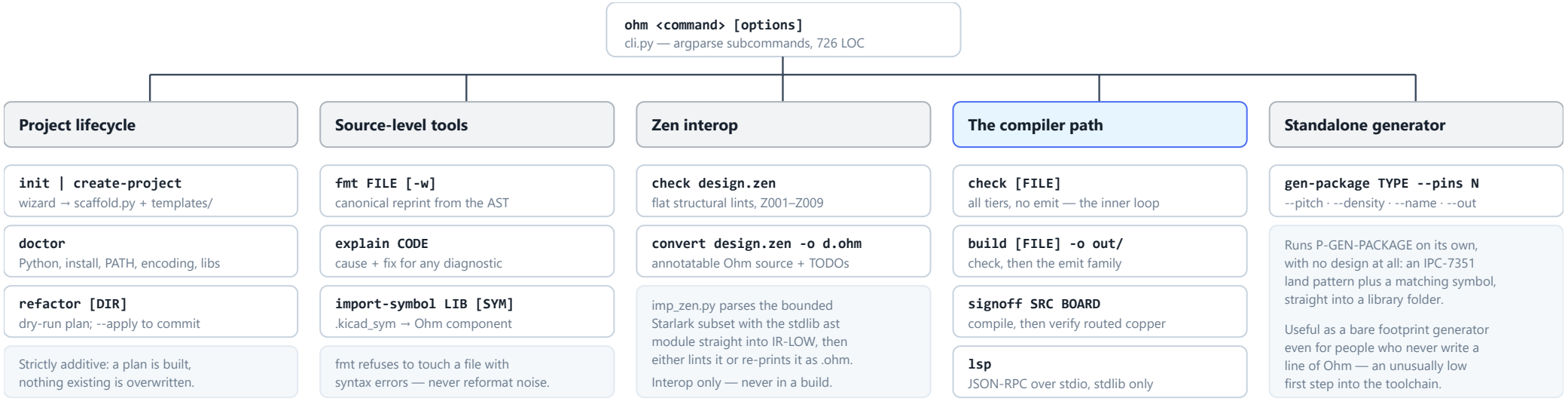
A part with a string footprint has no geometry the compiler may read — that would mean loading a KiCad library at build time and losing hermeticity. So the board gets a pad-less **stub**: correct LibId, position, `ohm_path`, reference and value. In KiCad, *Tools → Update Footprints from Library* hydrates every stub, and position and grouping survive.

The v1 package catalog

Gull-wing `SOIC SSOP TSSOP SOT23` · chip `CHIP0402 0603 0805 1206` · no-lead `QFN DFN SON` · quad `QFP` · grid `BGA LGA` · `PLCC` · lead-and-tab `SOT223 SOT89 TO252 TO263`. Each takes a density (`least` / `nominal` / `most`) selecting the IPC-7351 fillet set; anything outside the catalog is S006 at compile time.

Command surface and dispatch

One binary, five families of behaviour. Only the fourth family touches the compiler; the rest are deliberately kept off the pass pipeline so they stay fast and cannot corrupt a build. Inside a project, `build` and `check` need no arguments at all — `ohm.toml` supplies entry, output directory and target, framework-style.



Project discovery — a bare `ohm build` / `ohm check` walks up from the cwd to the first `ohm.toml` (project.py) and takes entry, out and target from it. An explicit FILE, `-o` or `--target-kicad` always wins — with all three given, the command is fully independent of any project at all.

One shared entry point — `compile_source(source, path, emit=…, debug_passes=…)`. `check` calls it with `emit=False` (the full tier stack still runs, nothing is kept); `build` with `emit=True`; `signoff` with `emit=True` and then re-verifies the routed board against the resulting IR-LOW; `lsp` re-runs it on every buffer change. Four commands, one truth.

FLAG	APPLIES TO	EFFECT
`--json-diagnostics`	check, build, signoff	Diagnostics as a JSON array (severity, code, message, file, help) — the machine channel agents iterate on, instead of scraping caret-annotated text.
`--strict`	check, build	Exit 1 on any warning or worse, *before* any emit. CI gate: the examples in this repo are enforced to compile clean under it.
`--emit <kind>`	build	`netlist · kicad · zen · constraints · package · eagle · all` (default `all`). `package` and `eagle` emit only the generated libraries.
`--dump-ir <level>`	build	`high · typed · low` — deterministic JSON of the chosen IR level, sorted keys. The debugging and golden-test surface.
`--debug-passes`	check, build	Re-serialises the design after every checker and raises if any pass mutated it. The non-mutation invariant, enforced at runtime.
`--target-kicad <n>`	build	KiCad major version to target; v1 supports 7. Also read from `ohm.toml`'s `target`.
`--ratings / --pindata`	build	Provider selection for pin ratings and pin tables. One provider (`declared`) ships today — the flag *is* the seam, kept explicit so a curated database can never become an implicit dependency.
`--git / --no-git / -y`	init	Whether to `git init` the new project, and whether to prompt at all. With no arguments on a terminal, `init` runs an interactive wizard and creates a named subfolder.

Why the families stay separate

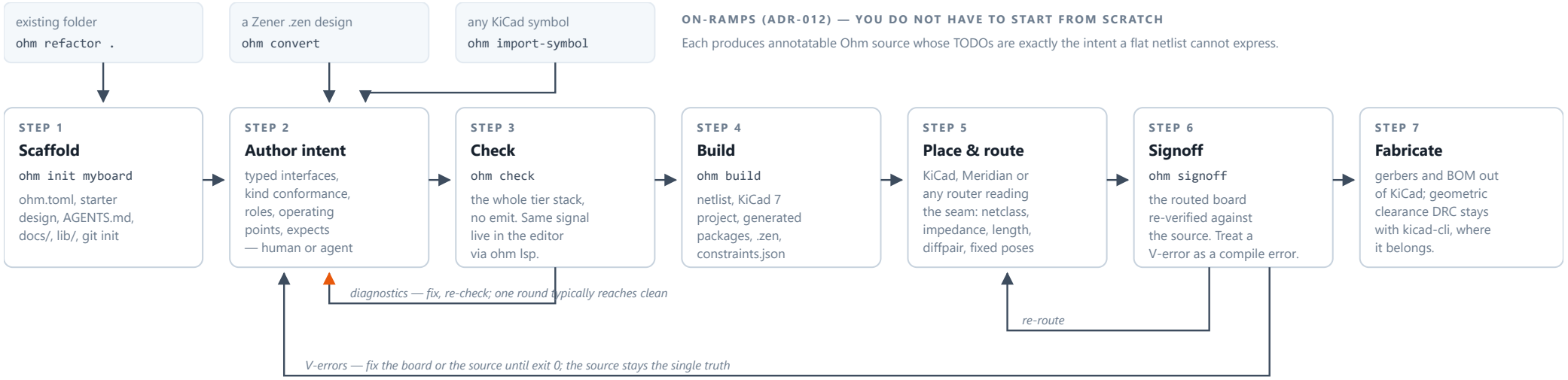
Keeping `fmt`, `explain`, `doctor` and the on-ramps off the pass pipeline is a reliability decision, not a stylistic one: none of them can be broken by a compiler bug, and none can leave a half-written artifact behind. `fmt` reprints from the AST and refuses a file that does not parse; `explain` reads a static table and needs no design at all.

The editor gets the same answers as CI

`ohm lsp` is a stdlib-only JSON-RPC server that runs the *same* `compile_source` on every `didOpen` / `didChange` / `didSave` and publishes the result as diagnostics. There is no second, weaker analysis path for the editor — a Tier-I miswire is a red squiggle before any build, and it is the identical diagnostic CI will produce.

Design lifecycle — the loop a board actually travels

Ohm closes the LLM→compile→route loop that open-loop agents leave open. Two feedback edges do the work: diagnostics send authoring back to the source, and signoff sends a *routed* board back to the source. Ohm meets designs where they already are — an existing folder, a `.zen` design or a KiCad symbol library are all on-ramps, not rewrites.



MEASURED, NOT CLAIMED — FROM THE V0.1.1 AGENT RUN REPORT (REAL_RUN_REPORTS/, 28 JULY 2026)

39 parts · 34 nets · 135 nodes

A USB-C powered STM32F103 board, 60 × 30 mm, written in 351 lines of Ohm plus a 199-line part library — driven end to end by an LLM agent.

60 constraints · 1 obligation

39 fixed, 12 near, 3 netclass, 2 diffpair, 2 impedance, 2 length — all recorded into constraints.json, plus 39 placements. The obligation left the compiler unsolved.

Byte-identical rebuild

The agent hashed the outputs, deleted out/ entirely, rebuilt and compared: .kicad_pcb, .net and .constraints.json all matched. Determinism, verified.

Caught before layout

Bring-up findings the netlist alone could not show — a missing crystal load cap (B008), an undriven input (I006) — each reasoned about the *net*, not the part.

What the loop guarantees

A build is reproducible forever: no network, no clock, no subprocess, no solver in the compiler, so the same source is the same bytes on any machine, any day. An errored design emits nothing at all. Every finding has a code, an explanation and a declared escape, so a waiver is an auditable note rather than a deletion. And the routed board is checked against the source it came from — which no board-local DRC can do.

What it deliberately does not claim

Undeclared intent is honestly unchecked — an un-kinded part, a pin with no rating and a `# TEMPLATE` stdlib block all say so rather than pretending. Geometric clearance is delegated to `kicad-cli pcb drc`; zones and mid-segment T-junctions are outside the signoff connectivity model (V005). Convergent physics is recorded as a Hole and solved downstream, never guessed in-compiler. And as of v0.1.1 no Ohm-designed board has come back from a fab and booted: the bring-up tier is test-validated, not field-validated.

The loop as commands

`ohm init myboard` → `cd myboard` → `ohm check` ↻ edit → `ohm build` → route in KiCad or Meridian → `ohm signoff myboard.ohm routed.kicad_pcb` → fabricate.

An agent runs the same loop with `--json-diagnostics` on every step and `ohm explain <CODE>` whenever a code is unfamiliar — which is the entire integration surface. There is no plugin API to learn, no GUI to drive, and no state carried between commands: every invocation is a pure function of the files on disk.

Repository map, invariants and seams

Where each pass lives, what must never change about how it behaves, and the three narrow interfaces through which anything outside the compiler is allowed to reach in.

MODULE MAP — ~8 000 LOC OF PYTHON, DEPENDENCIES: LARK (PARSING) AND THE STANDARD LIBRARY

STAGE	MODULES	LOC
front	ohm.lark grammar · parse.py · ast_nodes.py · loader.py	826
middle	resolve.py (symbols, resolution, elaboration) · ir.py (all three IR levels + deterministic JSON) · units.py	1322
lower	lower.py (P-CONNECT + P-LOWER) · holes.py	251
check	check_units · check_structural · check_kinds · check_intent · closedform · check_quant · check_bringup · check_mfg · ratings_iface	1668
emit	emit_netlist · emit_kicad · emit_kicad_pro · emit_zen · emit_constraints · emit_ohm	684
pkggen	packages (IPC-7351 catalog) · footprint · symbol · step · eagle · pindata_iface	1077
interop	imp_zen.py (bounded Starlark subset) · import_symbol.py	340
signoff	signoff.py — routed-board verification, pure over (IRLow, board text)	327
shell	cli.py · driver.py · diagnostics.py · lsp.py · fmt.py · project.py · scaffold.py · gitutil.py	1461
seam	shared/constraints_schema.py — the only open ↔ proprietary boundary	69
data	ohm/std/ 17 domain modules · ohm/templates/ scaffold templates · examples/ 6 complete boards	—
docs	docs/blueprint/ (read-only, canonical) · docs/adr/ ADR-002…012 · docs/STYLE.md · AGENTS.md · PROGRESS.md	—
tests	593 test functions across the tests/ tree — golden netlists and IR, adversarial known_bad/ suites, determinism gates, live-LLM authoring fixtures	—

The seam — shared/constraints_schema.py

Pure data, 69 lines, no computation on either side. ConstraintSet{version, constraints, holes, placement, groups}: R7 constraints with canonical source strings, recorded obligations, DSN-shaped placement triples and module-prefix part groups. Emitted by Ohm, consumed by the router and the convergence solver — which are separate products. The compiler never imports anything from the other side of this line, which is what makes "open language, proprietary computation" a structural fact rather than a promise.

Decision record — docs/adr/

002 zen interop subset · **003** closed-form semantics · **004** Tier-Q operating points · **005** constraint syntax · **006** pkggen catalog v1 · **007** stdlib · **008** kind system · **009** first spin · **010** repetition · **011** signoff · **012** expressiveness & ecosystem.

Each ADR records a decision and the alternatives it closed off, so a later change argues with the record rather than rediscovering it. docs/blueprint/ holds the three founding documents and is treated as read-only canon; PROGRESS.md is the execution log.

INVARIANTS — THE PROPERTIES EVERY CHANGE MUST PRESERVE

Hermetic (N1)

The compiler performs no network access, no clock reads, no subprocesses and no solver calls. Its *only* I/O is reading source files during the parse stage to follow use imports. Pin ratings and pin tables are frozen in the source by authoring time, reached through provider interfaces — so no build can ever depend on a remote catalogue being up.

Deterministic

Every collection is insertion-ordered or sorted by a stable key. Name uniquification uses a fixed scheme; net naming is order-independent. Arithmetic is Decimal, never float — the single square root in the package geometry is rounded to 0.01 mm before it re-enters Decimal world. IR dumps serialise with sorted keys; files are written UTF-8 with LF newlines on every platform, and the console streams are forced to UTF-8 so a cp1252 terminal cannot crash on a ΔV in a diagnostic.

Total, non-mutating analysis

Checkers never raise, never halt early and never write to the IR, so one run reports the entire error set and pass order cannot change the answer. --debug-passes re-serialises the design after every pass and raises if a byte moved. After a parse or resolve error the remaining analysis passes still run on the recovered partial design — but lowering and emit never do.

Provider seams, kept explicit

ratings_iface (Tier-Q ratings) and pkggen/pindata_iface (symbol pin tables) are interfaces with one shipped implementation each: read what the source declares. --ratings and --pindata exist today with a single choice precisely so that a curated database can never quietly become an implicit dependency of a build.

Honest about scope

Nothing claims to catch all bugs. Undeclared intent is unchecked and says so; un-kinded parts, # TEMPLATE stdlib blocks and the V005 delegation notice all exist to make the boundary visible. Diagnostics scale with declared intent — which is the whole design thesis, and also the honest reason a sparsely annotated design gets a quieter compiler.

Gates

pytest -q, ruff check . and mypy ohm gate every change, and CI enforces that the examples compile clean. Golden netlists and golden IR pin the output bytes; the known_bad/ suites pin the diagnostics; determinism tests rebuild and compare. The invariants above are not documentation — they are assertions in the suite.